

國立成功大學

工程科學系

碩士論文(初稿)

以 LLVM OpenMP Offloading 擴展儲存導向異質邊緣運  
算平台之設計

Design of a Storage-Oriented Heterogeneous Edge  
Computing Platform Extended with LLVM OpenMP  
Offloading

研究生：陳錦麟

指導教授：侯廷偉 博士

中華民國一十四年七月

# 摘要

隨著 GPU 及異質運算平台的普及，計算能力持續快速增長，但記憶體頻寬與延遲的瓶頸逐漸成為限制效能的關鍵因素，形成所謂的「記憶體牆（Memory Wall）」問題。尤其在大規模機器學習及資料分析任務中，資料頻繁在主機記憶體與加速裝置間搬移，帶來顯著的效能損失與能源消耗。

為了解決此問題，Computational Storage Device (CSD) 概念被提出，藉由將資料處理推至儲存裝置端，實現「資料就地運算」以減少不必要的資料搬移與降低頻寬需求。初期的 CSD 多聚焦於資料庫查詢加速，近年則逐漸演化為可程式化、支援複雜邏輯的運算儲存架構。

本研究提出一種基於 OpenMP offload 的軟體定義 CSD，結合 Linux-based 儲存裝置與多 NPU 資源管理，允許主機端以標準 pragma 語法進行運算卸載，有效緩解記憶體牆瓶頸，並提升資料運算效率。

**關鍵字：**

## **Extended Abstract**

## 致謝

# 目錄

|                                                |     |
|------------------------------------------------|-----|
| 摘要                                             | I   |
| Extended Abstract                              | II  |
| 致謝                                             | III |
| 目錄                                             | IV  |
| 表目錄                                            | VI  |
| 圖目錄                                            | VII |
| 第一章 緒論                                         | 1   |
| 1.1 研究背景                                       | 1   |
| 1.2 研究目的                                       | 1   |
| 1.3 研究貢獻                                       | 2   |
| 第二章 文獻探討                                       | 3   |
| 2.1 記憶體內存牆(Memory Wall)                        | 3   |
| 2.2 運算型儲存裝置(Computational Storage Device, CSD) | 3   |
| 2.3 LLVM                                       | 5   |
| 2.4 LLVM OpenMP                                | 7   |
| 2.5 神經網路處理單元(NPU)                              | 10  |
| 第三章 系統設計與實作                                    | 13  |
| 3.1 架構設計                                       | 13  |
| 3.2 SylphToolChain                             | 15  |
| 3.3 執行時動態函式庫的擴充與通訊方法                           | 16  |
| 3.4 環境配置                                       | 18  |
| 3.4.1 OpenMPI 配置                               | 19  |
| 3.4.2 OpenMP 配置                                | 20  |
| 3.4.3 OpenSSH 配置                               | 20  |
| 3.4.4 USB Gadget NCM 和 Mass Storage 驅動程式配置     | 21  |
| 第四章 實驗結果討論                                     | 23  |
| 4.1 實驗環境                                       | 24  |
| 4.2 Rodinia Benchmark：多核心與異質 CPU 效能比較          | 25  |
| 4.3 HeCBench：GPU 與異質 CPU 效能比較                  | 28  |
| 參考文獻                                           | 32  |



## 表目錄

|                             |    |
|-----------------------------|----|
| 表格 1、Sylph::Linker 生成之描述檔格式 | 16 |
| 表格 2、sylph.h 中的類型宣告與類型說明    | 16 |
| 表格 3、sylph.h 提供的函式宣告與功能說明   | 17 |

## 圖目錄

|                                                                  |    |
|------------------------------------------------------------------|----|
| 圖 1、LLVM OpenMP Offloading 編譯流程圖                                 | 8  |
| 圖 2、clang-linker-wrapper 內部的鏈結流程圖                                | 7  |
| 圖 3、LLVM OpenMP 調用 CUDA 裝置運算的動態鏈結圖                               | 8  |
| 圖 4、本論文架構下 clang-linker-wrapper 鏈結步驟圖                            | 14 |
| 圖 5、在 openmpi.mk 中新增 --enable-heterogeneous 開啟異構運算功能             | 19 |
| 圖 6、hwloc 2.7.0 中 hwloc.h 的 hwloc_topology_discovery_support 結構體 | 19 |
| 圖 7、hwloc 2.0.2 中 hwloc.h 的 hwloc_topology_discovery_support 結構體 | 20 |
| 圖 8、裝置端 /etc/init.d/S30usb gadget 腳本程式碼                          | 22 |



# 第一章 緒論

## 1.1 研究背景

近年來，隨著 GPU 等硬體加速器的快速演進，神經網路模型的發展亦隨之加速。GPU 原為圖形處理而設計，但其高度平行化的特性恰與神經網路中的矩陣運算需求相符，因而成為深度學習運算的主力。這一趨勢帶動了模型規模的不斷膨脹，模型參數與所需資料量呈指數級增長。

然而，硬體資源的成長速率相較之下已逐漸趨緩，無法完全因應神經網路日益龐大的運算與儲存需求。神經網路的發展與普及，已逐漸受限於硬體瓶頸，成為現今系統架構設計上的重大挑戰。

為突破此一限制，近年研究提出數種方向。例如，NPU (Neural Processing Unit) 作為專為神經網路而設計的運算加速器，因其更契合神經網路運算特性而逐漸取代部分 GPU 角色。另有研究嘗試將 GPU VRAM 擴充為類似 NAND Flash 的架構，以降低記憶體成本，同時提升模型儲存能力。另一方面，CSD (Computational Storage Device) 亦受到關注，其概念為將特定的資料密集處理任務（如裁剪、內積、矩陣乘法等）直接於儲存端執行，以減少 CPU 與儲存裝置之間的資料搬移，顯著降低資料處理延遲與能耗。

## 1.2 研究目的

本論文認為，NPU、記憶體擴充設計與 CSD 這三類解決方案並非彼此排斥，反而具整合潛力，能形成一種更高效的異質運算架構。因此，本文提出一種基於儲存裝置內運算的平行處理開發介面，其核心理念為：將儲存裝置與運算加速單元整合為類 CSD 架構，透過硬體層級優化資料讀取與模型運算流程，並將中央處理器資源留給使用者互動與控制任務，以達成運算效率最大化。

具體而言，本研究以具備 CPU 的嵌入式裝置（代稱 Sylph）作為儲存主體，由其 CPU 同時做為儲存裝置控制器以及異質運算加速器，形塑出具備邊緣運算特性的

智慧型儲存模組。並在實驗中額外整合 NPU 負責神經網路等運算密集型任務，分析特定應用下 CPU 與 NPU 之間的任務分工與資料交換效能，進一步驗證異質運算架構在資料密集型場景下的可行性與效率提升。

與多數 CSD 僅能處理靜態資料流程不同，本文進一步整合 LLVM OpenMP 編譯架構，提供開發者彈性且通用的任務卸載介面。OpenMP 擁有強大的工作併行指令與任務指派機制，允許開發者將細碎或分散的資料處理任務卸載至周邊裝置執行；而在 Sylph 架構上，更可活用資料就近獲取的特點，使整體裝置運作更加高效與自主。

### 1.3 研究貢獻

本研究針對當代異質運算與資料密集型應用需求，提出一套結合運算與儲存功能的 CSD 周邊裝置設計方法，並以實作驗證其於實際應用場景中的可行性與效能潛力。整體研究貢獻可歸納如下：

1. 嘗試設計並實作一套結合嵌入式運算單元與儲存裝置功能的 CSD 架構，透過 USB gadget 與以太網通訊方式，使其可作為主機系統之擴充運算節點與儲存節點，提供可嵌入式部署且具備運算能力的邊緣裝置實作基礎。
2. 為突破傳統 OpenMP target 模型僅支援特定平台的限制，本研究對 LLVM OpenMP 編譯與鏈結流程進行調整與擴充，使經 Buildroot 構建之輕量級 Linux 裝置能被視為合法之 offload 運算目標，並能透過 OpenMP target 介面實際承接主機下發之運算任務。
3. 透過多組 benchmark 與深度學習推論任務，針對 CPU、GPU 與所提出之 CSD 架構進行效能分析與比較，評估其在不同資料與運算分工配置下是否能保留異質裝置各自優勢，甚至互補不足，並著重於傳輸延遲與裝置計算能力之平衡關係。
4. 在整合過程中，發現現有應用程式在記憶體模型、inline 函式、鏈結機制與資料指標語意上，與 OpenMP target 模型存在相容性障礙，並提出對應修正策略與工具鏈封裝方法。這些發現提供未來開發以 LLVM OpenMP 為基礎的異質 offload 裝置設計時之實務參考。

## 第二章 文獻探討

### 2.1 記憶體內存牆(Memory Wall)

記憶體牆議題最早在 1995 年被 Wulf 和 McKee 提出，當時中央處理器速度持續提升，而記憶體存取速度的提升卻跟不上，導致記憶體成為整體系統瓶頸，而歷年來有許多解決方法被開發出來：透過對快取作分層來增加快取命中率，進而減少 DRAM 存取次數；使用非一致性記憶體存取架構，靠拓譜縮短資料傳輸延遲；透過預測資料需求，提前將資料傳入快取中。

然而，在神經學習模型盛行的現在，個人電腦以及資料中心伺服器開始採用 CPU + GPU 的異質運算架構，原先在 CPU 上遇到的記憶體牆問題陸續在 GPU 上出現。文獻[1]中提到，現今模型參數需求量以每兩年 750 倍的趨勢增長，在硬體上，GPU 記憶體成長只有 2 倍，而通訊頻寬成長也只有 1.4 倍，導致 GPU 和主記憶體資料傳輸頻繁。記憶體牆議題已經從傳統 CPU 跟主記憶體，擴展到了 GPU 和主機端資料存儲裝置(Cache, DRAM, NVMe) 的層級管理問題，過去關於記憶體牆的解決方法在 GPU 上並不完全適用。

記憶體牆不只影響了神經網路模型的訓練成本。隨著 GPU 記憶體大小的增長，也會帶動新的、更大量參數的模型的研究與開發，這意味著神經網路模型的發展也與記憶體牆密切相關。

在模型上，利用參數量化以及模型剪枝來降低模型大小事普遍且高效率的方法，但兩者對模型準確率會帶來影響；文獻[2]嘗試利用低延遲快閃記憶體取代 GPU 記憶體，進而降低 GPU 記憶體成本；文獻[3], [4]透過預測模型預測的運算路徑，將暫時不需要的模型參數卸載到 DRAM 和 NVMe 上，以提高記憶體使用率。

### 2.2 運算型儲存裝置(Computational Storage Device, CSD)

運算型儲存裝置(CSD) 是一種內建通用、專用或可重構處理器的儲存裝置，能夠在資料本地執行原本由主機負責的運算任務，進而降低資料搬移開銷，提升系統

整體效能與能源效率。隨著資料量快速成長，傳統儲存裝置與主機間大量資料搬移所帶來的效能與能耗瓶頸日益明顯，促使 CSD 的概念逐漸成熟。[5]

圖 3 介紹了典型的 3 種 CSD 架構：

1. **外接運算單元**：將運算單元與 SSD 分開，使得主機、運算單元和 SSD 共用網路，但利用 DMA 等技術使得運算單元與 SSD 間的資料傳輸不透過主機完成，以達到降低主機運算資料和主記憶體開銷。
2. **內建運算單元**：將運算單元整合進 SSD 之中，使得 SSD 與運算單元有著獨立的資料傳輸網路，在硬體開發上能最佳化特定應用上的資料傳輸，發揮進資料端運算優勢。Samsung SmartSSD 便是此種架構，提供 FPGA 燒錄功能讓開發者在開發儲存裝置讀取介面時能將工作卸載到 FPGA 上。
3. **複數 SSD 串接在運算單元上**：讓運算單元成為多個儲存裝置的管理單元並與儲存裝置有著獨立的資料傳輸網路，該架構能讓資料傳輸不佔據主機與周邊裝置網路，且有著如外接運算單元一樣對 SSD 增減的彈性。

CSD (Computational Storage Device) 結合了運算單元與儲存單元，並將其封裝為模組化的開發架構 (programming model)，提供開發者針對主機介面進行功能擴充與應用設計。根據文獻 [5] 的分類，CSD programming model 可分為三種類型：

1. **Dataflow programming model**：此模型提供功能安裝 (attach/detach) 與資料流定義 (define dataflow) 機制。開發者可將特定應用功能部署於裝置端，並重新定義資料流規格，使裝置能根據資料內容進行結構解析與自動觸發對應運算邏輯。
2. **Server-Client programming model**：裝置提供安裝功能與指令觸發機制，主機端透過發送請求 (例如使用 vendor-defined command 或模擬 RPC 的封包格式) 呼叫裝置執行特定功能，常搭配封裝過的 API 使用，提升應用程式整合性。
3. **Shared-Memory programming model**：裝置與主機共享一致性記憶體區域，開發者得以如同撰寫多核心程式一般開發異質應用。儘管此模型提供最直觀的程式設計體驗，但其實現仰賴特殊硬體支援，例如支援一致性共享記憶體的 PCIe interconnect 技術，以及跨 ISA 的虛擬執行環境 (如 Popcorn Linux 或 H-Container)。

上述三種模型主要面向開發者，而非終端使用者。與 GPU 的通用性與高度程式性相較，CSD 目前仍以特定應用導向為設計核心，其高效能表現往往建立在「資料格式已知」、「儲存方式已知」的假設之上，開發者藉由限縮使用者對儲存裝置的操作來最佳化特定應用，將功能改以硬體方式實現，燒錄進硬體加速器後封裝於介面中，或是規範內部資料儲存格式，例如關聯式資料庫、向量資料庫等，藉此讓查詢與處理流程能跳過傳統的全表遍歷（full scan），直接透過在裝置上進行資料索引與向量空間距離計算等方式，實現近似即時的回應速度。

然而，這種設計也反映出目前 CSD 發展上的一項侷限——即尚未能提供如 GPU 一般的通用計算能力與靈活的開發環境。多數 CSD 的計算資源仍集中於特定任務，如壓縮解壓、篩選、排序或相似度比對，而缺乏完整的通用程式語言支援與動態工作調度能力。儘管如此，隨著資料生成與儲存量持續成長，針對應用進行資料格式與存取路徑共設計所能帶來的效能提升，使得 CSD 在特定領域如機器學習前處理、資料庫查詢加速或儲存端推論中展現出顯著潛力。

## 2.3 LLVM

LLVM（Low Level Virtual Machine）[6]是一個模組化且可重複使用的編譯器基礎架構，旨在支援語言前端與後端之間的清晰分離與擴充彈性。其獨特之處在於將編譯過程劃分為前端、中端與後端三大模組，使得各語言可以共用優化與後端機制。例如，Clang 作為 LLVM 的 C/C++/OpenMP 前端，能將原始程式碼轉換為 LLVM 中介表示（LLVM IR），再由後端根據目標架構生成對應機器碼。

在 LLVM Clang 中，工具鏈的管理是透過 ToolChain 類別實現。每個工具鏈（例如 CUDA、GNU）皆透過繼承 ToolChain 類別的方式來定義其行為。ToolChain 類別中會使用一系列繼承自 Tool 類別的元件來表示實際的編譯階段工具，例如前處理器、編譯器、組譯器與鏈結器等。這些工具元件透過 ConstructJob 方法建構對應階段的執行指令（如 clang-cc1 或 ld），並依據目標平台、語言參數與使用者指定選項生成完整的命令執行流程。這種設計不僅提供高度擴充性，也使得 Clang 能夠被廣泛應用於嵌入式系統、異質計算與跨平台開發等場景。

在 LLVM 架構中，Clang 的核心元件之一是 Driver 類別，負責管理整體編譯流程。Clang Driver 接收使用者提供的編譯參數與輸入檔案，並透過 BuildCompilation() 方法建構出一個代表完整編譯任務的 Compilation 物件。該物件記錄了整體編譯過程所需的所有指令（Jobs）、執行順序與輸出規則，作為後續執行編譯的基礎。

在 BuildCompilation() 的執行過程中，Driver 會先呼叫 getToolChain() 根據目標三元組（TripleTarget）選擇適當的工具鏈，例如 GNU、Clang、或客製化工具鏈。接著，Driver 會透過 BuildActions() 建立邏輯上的編譯階段圖（Action DAG），其中每個 Action 物件代表一個獨立的編譯步驟，如 InputAction、CompileJobAction、AssembleJobAction 或 LinkJobAction 等，形成完整且有向無環的編譯計畫。

最後，Driver 會呼叫 BuildJobs()，根據前述的 Action 圖與所選定的 ToolChain，將每個 Action 轉換為具體可執行的 Job。這些 Jobs 會透過 ToolChain 中的 SelectTool() 方法選擇對應的 Tool（如編譯器、組譯器、鏈結器），並由 Tool::ConstructJob() 方法建構實際執行的命令（Command），最終將整個編譯流程具體化為一連串可執行的系統指令。

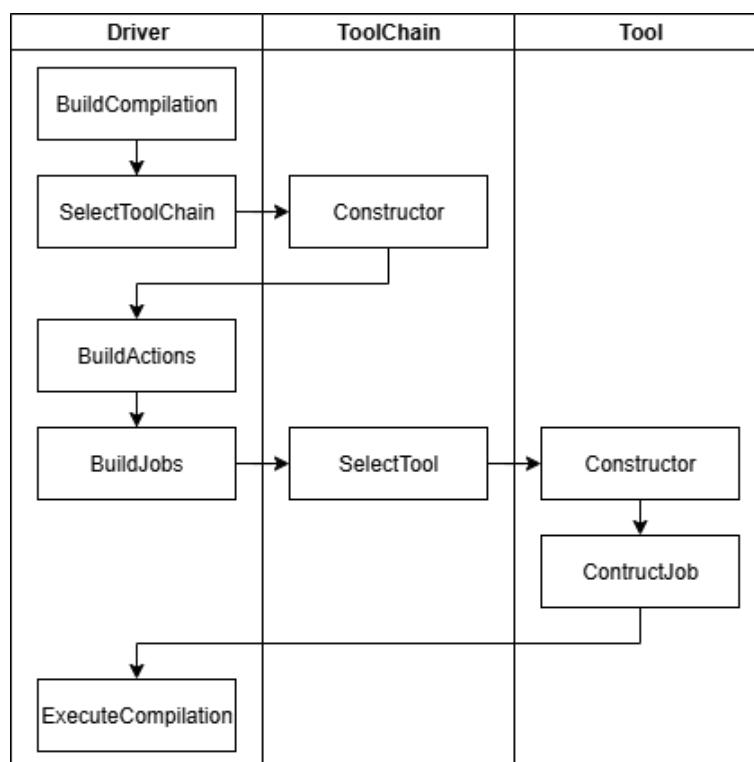


圖 1、LLVM Clang 編譯工作流程圖生成流程

## 2.4 LLVM OpenMP

LLVM OpenMP 是 LLVM 編譯器中提供的 OpenMP 支援，由於 OpenMP 應用程式是藉由在程式碼中加上 `#pragma omp <option>` 進行開發，而非單純加入 `omp.h` 函式庫，編譯器必須從前端加入 OpenMP 語法分析功能。

OpenMP 應用程式在編譯時會使用 `-fopenmp` 參數開啟編譯器中的 OpenMP 支援，在 LLVM 4.0 ( 2017 ) 擴充並開始支援 OpenMP Offloading。OpenMP Offloading 是 OpenMP 為了支援異構運算而擴充的功能，透過在編譯時加上 `-fopenmp-targets` 參數，程式碼中 `#pragma omp target` 作用域內程式（以下簡稱卸載程式）將在執行時被卸載到異構運算裝置上執行。

在編譯架構上，LLVM OpenMP Offloading 順應了 LLVM 的核心理念進行開發，Driver 在 `BuildCompilation()` 中會根據 `-fopenmp-targets` 找出卸載程式要使用的裝置工具鏈，而 `BuildActions()` 階段則會使用 `OffloadAction` 在原編譯流程上生成一條編譯卸載程式用的 Action 路徑，卸載程式內的程式碼會被分離出主程式後組譯成異構設備看得懂的機器碼，再由 `LinkerWrapperJobAction` 負責進行鏈結與嵌入進主程式中。

圖 3 是追蹤了 clang 編譯 LLVM OpenMP Offloading 的編譯流程所繪製出的 Action DAG，因為鏈結階段與主、卸載程式的整合被包裝在了 `clang-linker-wrapper` 執行檔中，所以圖 2 額外描述了鏈結階段流程。圖 3 左邊路徑上，從 `InputAction` 到 `CompileJobAction` 是 `OffloadAction` 生成的空路徑，不會產生 Job。

Clang-linker-wrapper 利用 `clang` 指令尋找裝置工具鏈並執行卸載程式的鏈結階段，主程式則是根據 `--linker-path` 參數或者系統環境選擇鏈結器，在主程式鏈結時同時將卸載程式檔案內嵌進主程式檔案中。

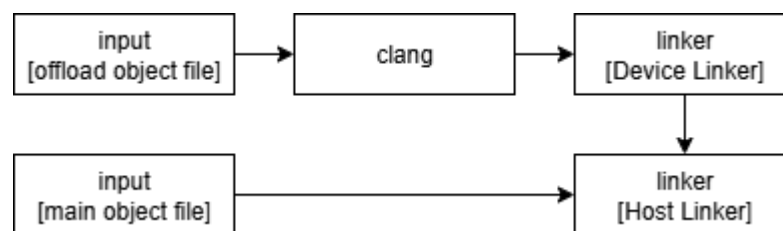


圖 2、clang-linker-wrapper 內部的鏈結流程圖

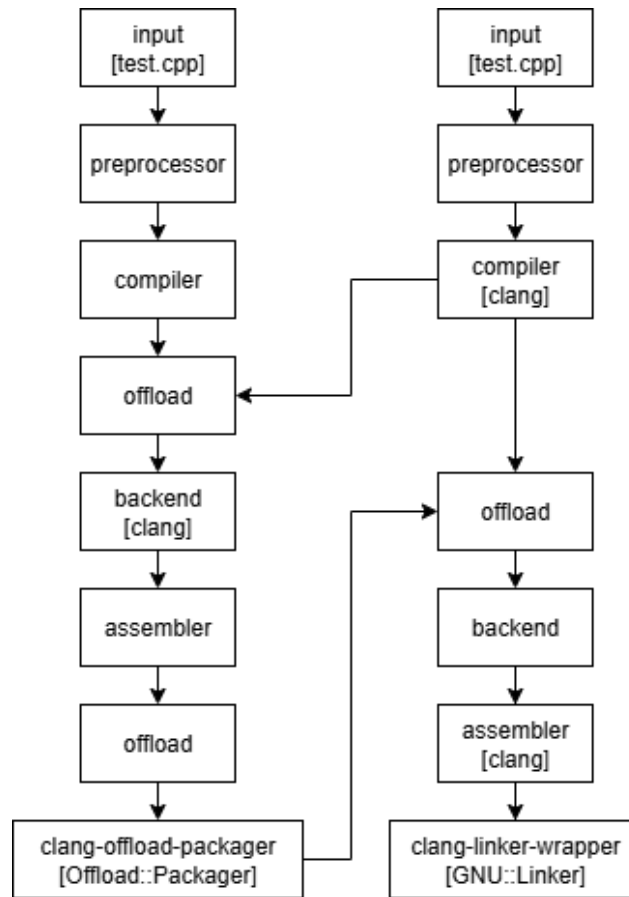


圖 3、LLVM OpenMP Offloading 編譯流程圖

OpenMP Offload 應用程式執行後，會動態加載 libomptarget.so，它將提供設備管理、數據傳輸、變數映射、工作卸載等一系列服務來支持編譯架構定義出來的方法與介面。圖 1 描述了以 CUDA 裝置為例的 libomptarget.so 動態調用關係。其中，libLLVMOffload.so 內的 libomptarget.rtl.cuda 模組會作為中間橋梁，使用 CUDA Toolkit 內的應用程式開發介面來定義 omptarget.so 提供給 OpenMP Offload 應用程式的函式。



圖 2、LLVM OpenMP 調用 CUDA 裝置運算的動態鏈結圖

LLVM OpenMP Offloading 提供 libomptarget.so 動態函式庫實現應用程式開發界面，在 LLVM 20.0 之前，libomptarget.so 會動態載入 libomptarget.rtl.cuda.so，libomptarget.rtl.amdgpu.so 等運行期函式庫來與對應的顯示卡 API 對接。在 LLVM 20.0 之後，運行期函式庫被整合成了 libLLVMOffload.so，而相關程式碼被移至



offload/plugins-nextgen 目錄底下，並保持與以往相同的軟體架構。

LLVM OpenMP 採用多型設計，自定義模組需要定義多個繼承自原有類型的子類型來完善擴充功能。當應用程式啟動時，會實例化 LLVMOffload 內的所有 Plugin 類型，PluginTy 類型要檢查所有可存取裝置。緊接著程式會迭代所有嵌入之卸載工作映像檔，根據相容性選擇對應的 PluginTy 建立 DeviceTy 類型並將映像檔載入裝置中，並利用 DeviceImageTy 類型管理。Plugin 會利用 GlobalHandlerTy 類型取得映像檔中每個 #pragma omp target 程式片段的起始地址，並在該任務執行時實例化 Kernel 類型後請求 Kernel 開始執行。

LLVM OpenMP 工作卸載分為 3 個階段，如圖 2 所示，ProcessDataBefore 階段會為主程式與卸載程式共用的變數(使用 #pragma omp target map 宣告) 映射一空間到裝置端後將數據傳到裝置上；執行階段會將卸載程式中所有未宣告變數當作參數，若變數在 ProcessDataBefore 階段有映射裝置空間，則會用相應的裝置指標代替，好讓變數值能回傳回主機端；ProcessDataAfter 階段會將有映射且使用 map(from)宣告之變數值回傳回主機端。

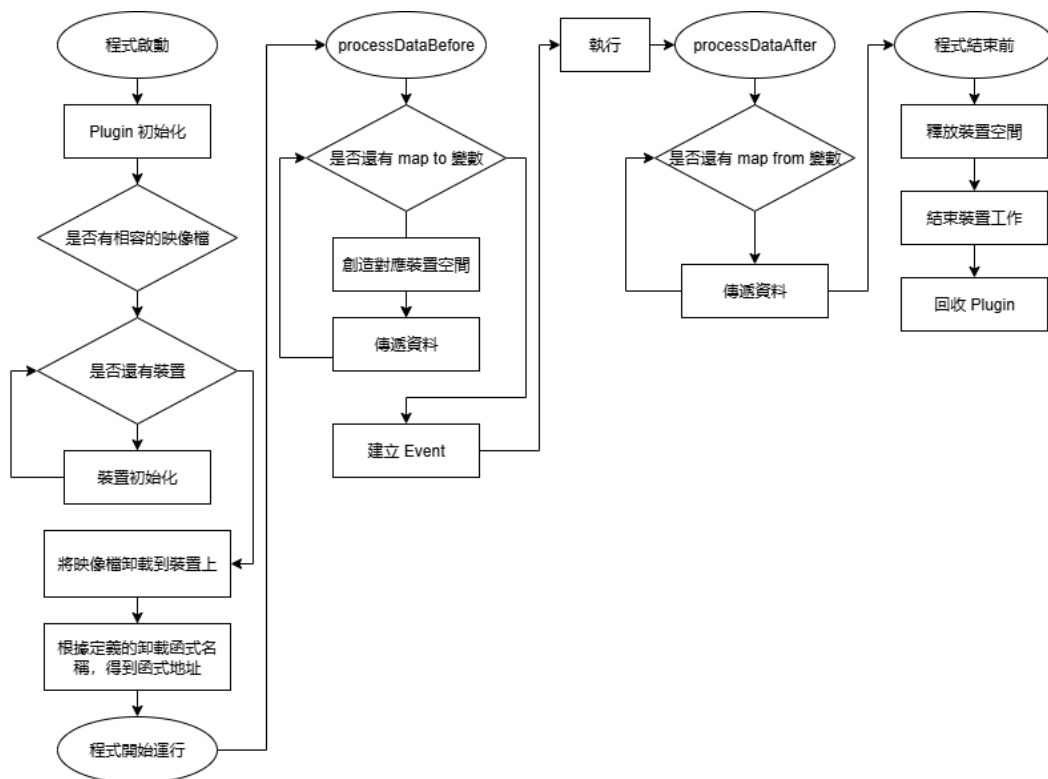


圖 3、OpenMP Offload 執行時程式卸載和資料傳輸流程

文獻 [7] 在 libomptarget.so 和 libomptarget.rtl.cuda.so 中間加上了網路通訊系統，讓工作可以藉由網路卸載到其他節點設備上。本論文認為，該文獻不僅實現利用 OpenMP 開發分布式運算功能，也展現了將工作卸載到基於作業系統之裝置上的可能性。

儘管主流異構運算裝置提供的數據傳輸介面是以 Memcpy 呈現，顯而易見地，數據在物理層面上依舊必須被傳輸到裝置記憶體上運算。換句話說，OpenMP Offload 在物理層面上已經拋棄了 OpenMP 強調的 Share Memory 理念，記憶體牆（Memory Wall）議題便隨之而來，文獻[8]在文獻 [7] 的基礎上改用 OpenMPI 做網路通訊，OpenMPI 的去中心化機制讓遠端節點可以繞過主節點做數據傳輸，並利用 Nvidia 提供的 GPUDirect 服務加快節點內的數據傳輸速度。文獻 [9] 則藉由讓同節點內多核心合作，來降低 OpenMPI 傳輸時間。

本論文將基於文獻[8]的方法，以 OpenMPI 作為通訊方法去擴充 LLVM OpenMP Offload，讓 LLVM OpenMP 支援編譯出使用本論文硬體架構 Sylph 卸載工作之應用程式。

## 2.5 神經網路處理單元(NPU)

隨著深度神經網路（Deep Neural Networks, DNN）在各種應用中展現卓越表現，例如圖像辨識、語音辨識、自然語言處理等，對於計算資源的需求也迅速增加。早期的神經網路運算主要依賴 GPU（Graphics Processing Unit），由於其大量並行運算核心的設計相當適合處理矩陣乘法與卷積等運算，因此一度成為主流的 AI 訓練與推論平台。

然而，隨著部署場景擴展到邊緣裝置與物聯網應用，GPU 逐漸暴露出以下幾點限制：運作時耗能相對較大，不利於電池供電或被動散熱的系統；在高負載下運行的 GPU 容易產生大量熱能，需要額外散熱機制；GPU 多數設計針對桌機或伺服器，體積龐大對嵌入式系統不夠友善。

在這樣的背景下，**神經網路處理器（Neural Processing Unit, NPU）**應運而生。

NPU 是專門為神經網路推論加速所設計的硬體加速器，透過針對性的架構設計，在不犧牲效能的前提下，有效降低功耗與硬體資源佔用，特別適合部署於嵌入式系統或邊緣裝置。

文獻[10]將現今的 NPU 分類並統整出較常見的三大類：

### 1. 資料流架構 (Dataflow Architecture)

資料流架構以最大化資料重用並最小化記憶體訪問為設計核心。其透過專用的計算排程與硬體路徑，使神經網路中不同的張量（例如輸入、權重、輸出）能在硬體層上高效流通。

代表性實作如 Google TPU 所採用的 systolic array 設計，以及 MIT 開發的 Eyeriss 架構等，皆屬此類。這類架構常具備可預測且高吞吐的計算性能，特別適合 dense CNN 等規則模型。

### 2. 記憶體內運算 (Processing-In-Memory, PIM)

記憶體內運算 (PIM) 架構試圖將部分運算邏輯直接整合至記憶體模組中，藉此減少資料於記憶體與處理器間的搬移開銷。此類設計通常結合特殊的儲存技術（如 ReRAM、MRAM），並直接在記憶體中執行向量乘法、加總等基本運算。

PIM 架構在處理大規模神經網路參數或中介特徵時，能有效降低能耗與延遲，近年亦被應用於儲存計算整合的設計理念中。

### 3. 類神經形態計算 (Neuromorphic Computing)

類神經形態計算 (Neuromorphic Computing) 以生物神經系統為靈感，透過模擬神經元與突觸間的事件驅動式訊號傳遞進行運算。此類系統多採用 Spiking Neural Networks (SNN) 作為演算法基礎，並設計相應的非同步運算硬體。

代表實作如 IBM TrueNorth 與 Intel Loihi，皆以極低功耗的運作特性著稱，特別適合時間序列處理、感測融合與自主系統等即時性應用場景。

在部分運算行儲存裝置上也會使用硬體加速器進行神經網路運算，文獻 [11] 中提到在圖像儲存系統中，會使用神經網路從資料生成特徵向量後進行管理，藉由儲存特徵向量當作索引值，系統可以使用向量檢索算法等圖檢索方法快速找到特定特徵或是與特定圖像相似的已儲存圖像。

文獻[11]將特徵向量的生成從主機端搬到固態硬碟的硬體加速器 DLG-x 上，減少了 69.9% 的運算延遲、34.4 % 的 CPU 耗能以及 63.0% 的 GPU 耗能。

本論文將參考論文 X 所以提出多 NPU 邊緣設備系統設計，將 Dataflow NPU 視為 CSD 資源，並以函式宣告提供特定功能介面給主機端 OpenMP Offload 應用程式開發，來比較在特定應用上本論文架構的表現。

## 第三章 系統設計與實作

### 3.1 架構設計

本系統旨在建構一個結合異質運算與儲存功能的計算儲存裝置 (Computational Storage Device, CSD)，使得主機端可透過 OpenMP target pragma 將運算卸載至儲存裝置內部的嵌入式處理器執行，進而減少資料傳輸負擔，達到資料近端處理 (near-data processing) 的目的。整體系統由三個主要元件組成：

1. 主機端：負責發起編譯、應用程式執行時的動態連接庫支援。
2. 裝置端：接收主機端的工作卸載請求，執行卸載程式，並提供資料儲存功能。
3. 通訊層：負責主機端與裝備端的資料通訊和同步管理。

為達此目標，本研究採用 LLVM/Clang 編譯器架構，並設計一套客製化工具鏈 SylphToolChain，以支援主機端對裝置端之 OpenMP offloading 編譯與執行流程，其目標為使用外部提供的 GNU cross-toolchain 對 target pragma 進行編譯。此工具鏈的核心改動包含：

1. 前處理和編譯階段：保留 Clang 預設行為，無須改動。
2. 裝置端組譯階段：呼叫 GNU cross-toolchain 的組譯器進行組譯。
3. 裝置端鏈結階段：使用 GNU cross-toolchain 的 GCC 進行鏈結，並且將鏈結後的映像檔儲存至裝置端。接著提供描述檔嵌入回主程式中。

關於鏈結階段的改動，將映像檔直接儲存至裝置端，以分離主程式和卸載程式。這麼做不僅能降低主程式檔案大小進而減少程式在主機端的記憶體負擔，還能降低應用程式執行時把卸載工作傳送到裝置端的時間。

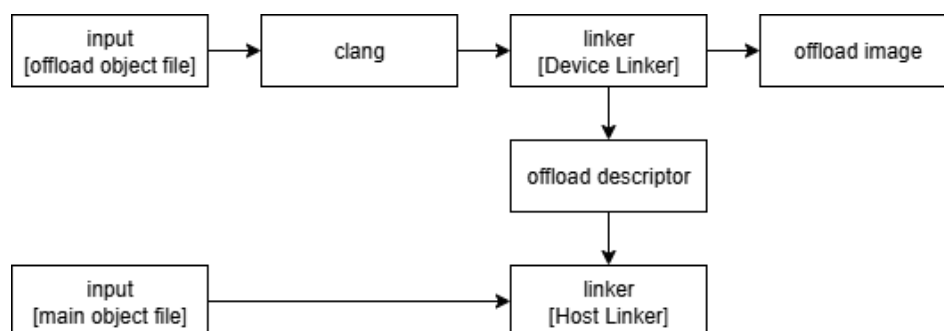


圖 4、本論文架構下 clang-linker-wrapper 鏈結步驟圖

根據本論文第 2.4 章節對異質裝置模型的分析，為了實現 OpenMP 應用程式於執行階段能將運算卸載至儲存型裝置端，本研究擴充 LLVM OpenMP Offloading 架構中的動態函式庫，實作出新的 libomptarget.rtl.sylph 模組。

該模組負責建立 libLLVMOffload.so 與裝置端之間的通訊橋接，透過本研究所設計的通訊層，實作完整的 offload 作業流程，包括：裝置初始化、binary 傳輸、記憶體管理、kernel 啟動與結果回傳等功能。

為簡化主機與裝置端之間的溝通機制並確保高可擴展性與效能，本系統通訊層採用 OpenMPI 作為底層通訊架構。相較於自行實作 socket-based 或共享記憶體等傳輸機制，OpenMPI 提供了成熟且高度最佳化的訊息傳遞介面，可大幅降低通訊層的開發複雜度，並具備以下優點：

1. **跨平台與跨架構支援**：OpenMPI 支援異質架構下的節點間溝通，能無縫地連接 x86\_64 主機與 ARM 架構的嵌入式裝置。
2. **高階抽象 API**：開發者可透過如 MPI\_Send、MPI\_Recv、MPI\_Comm\_spawn 等高階 API 操作資料傳輸與行程管理，降低錯誤率並加快開發流程。
3. **模組化整合**：OpenMPI 的 API 可直接嵌入 libomptarget.rtl.sylph 模組中，作為卸載 runtime 的一部分，確保整體架構一致性。

本系統將在應用程式啟動時初始化 MPI 環境，libomptarget.rtl.sylph 會透過 MPI\_Comm\_spawn 在裝置上開啟通訊層行程 worker，worker 將負責回應來自主機端的請求、與主機端進行資料傳送，以及管理卸載程式。

以下，將會依序介紹本架構的實作細節，首先，3.2 章節會介紹 SylphToolChain 的實作方法。接著，3.3 章節會介紹 libomptarget.rtl.sylph、worker 以及兩者之間的通訊層介面。最後，將說明主機端開發環境的必要套件配置，以及如何透過 Buildroot 的 menuconfig 介面，選擇並啟用裝置端系統所需的功能模組，以建立支援 OpenMP offloading 與遠端通訊的嵌入式 Linux 作業系統。

## 3.2 SylphToolChain

為使 LLVM OpenMP 能夠將卸載區塊編譯為可於裝置端執行的機器碼，根據本論文第 2.3 與第 2.4 節所述，需準備裝置端對應架構的跨平台工具鏈，並於 Clang Driver 架構中實作一套客製化工具鏈 SylphToolChain，使其能正確識別並套用該工具鏈於 offload 編譯階段。

本研究選用 Nuvoton MA35D1 嵌入式開發板作為裝置端，其開發商提供 Buildroot-based 系統建構機制，允許透過 make sdk 指令快速擷取包含 GNU 工具鏈與 sysroot 的開發套件。本系統即以該 SDK 為基礎，建構出針對 AArch64 架構之跨編譯工具鏈環境。

SylphToolChain 類別設計上參考 LLVM 內建的 GNUToolChain 實作，定義 GCCInstallationDetector 類別，負責於工具鏈初始化階段，自動搜尋與辨識所需的目標 GNU 工具與 sysroot 路徑。此機制確保 Clang 在執行 target 編譯階段時，能正確調用對應架構之編譯器與相關資源。

此外，為完成卸載映像檔的生成流程，SylphToolChain 中亦實作自定義工具類別 sylph::Assembler 與 sylph::Linker，兩者皆繼承自 LLVM 中的 Tool 抽象類別，並分別覆寫其 ConstructJob 方法。sylph::Assembler 用於生成目標 GNU 工具鏈格式之組譯指令，sylph::Linker 則負責生成卸載端可執行映像檔之鏈結指令。

特別地，`sylph::Linker::ConstructJob` 除了完成鏈結動作外，亦額外調用 `/bin/sh` 產生一份卸載映像檔之描述檔（descriptor file），該描述檔將作為中介，於後續主程式鏈結階段取代實際映像檔被嵌入主執行檔中，進而實現映像檔與主程式的分離配置策略。表 x 為描述檔格式，開頭使用特殊字串作為讓 LLVM OpenMP 辨認其與 `libomptarget.sylph.rtl` 相容，其後為卸載映像檔在裝置端之路徑。

表格 1、Sylph::Linker 生成之描述檔格式

|           |                    |
|-----------|--------------------|
| ##Sylph## | Offload Image Path |
|-----------|--------------------|

### 3.3 執行時動態函式庫的擴充與通訊方法

`libomptarget.rtl.sylph` 參考了 `libomptarget.rtl.host` 映像檔之動態載入實作方法，將所有 `#pragma omp target` 程式片段各自編譯成函式，執行期間使用 `ffi` 函式庫動態呼叫，所有參數會合併在依連續空間後使用 `ffi_call` 呼叫目標函數地址的函數，函數會根據指標與編譯時決定的順序取得對應參數。

主機端與裝置端的行程透過 OpenMPI 進行管理，在呼叫 `libomptarget.rtl.sylph` 內的 `SylphDeviceTy::init()` 時，使用 OpenMPI 在裝置端開啟卸載任務管理行程 worker，`libomptarget` 將透過 `MPI_Send` 向 worker 發送請求，亦或是利用 `MPI_Send` 及 `MPI_Recv` 相互傳遞資料。

為了架構維護，本架構參考 `cuda` 實作主機端請求介面 `sylph.h`，並將 OpenMPI 函式包裝在 `sylph.cpp`，降低 OpenMP Plugin 與 OpenMPI 的耦合性。`Sylph.h` 中定義了表格 a 中的類型，`libomptarget.rtl.sylph` 將透過 `sylph.h` 使用表格 b 中的函式。

表格 2、`sylph.h` 中的類型宣告與類型說明

| 定義類型                     | 實際類型                  | 類型說明                                                                     |
|--------------------------|-----------------------|--------------------------------------------------------------------------|
| <code>sy_device_t</code> | <code>uint32_t</code> | 裝置類型， <code>sylph.cpp</code> 可以藉由此類型得到該裝置的 OpenMPI Communication Rank ID |
| <code>sy_dl_t</code>     | <code>uint32_t</code> | 映像檔類型，worker 中每個動態鏈結的映像檔有獨立的 <code>sy_dl_t</code> 。                      |
| <code>sy_ptr_t</code>    | <code>uint32_t</code> | 指標類型，當主機在 worker 上分配空間時，會回傳一個 <code>sy_ptr_t</code> ，本指標不能在主程式中使用。       |



表格 3、sylph.h 提供的函式宣告與功能說明

| 函式名稱              | 參數                                                                                   | 函式功能                                                           |
|-------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------|
| sy_init           |                                                                                      | 呼叫 OpenMPI 初始化函數                                               |
| sy_finalize       |                                                                                      | 結束所有裝置的 worker 行程並呼叫 OpenMPI 終止函數。                             |
| sy_quit           | dev: sy_device_t                                                                     | 結束 dev 裝置上的 worker 行程。                                         |
| sy_get_symbol_ptr | ptr: sy_ptr_t&<br>size: int&<br>name: const char*<br>dev: sy_device_t<br>dl: sy_dl_t | 取得 dev 裝置上 dl 映像檔中，name 全域變數的變數地址，在 OpenMP 中用來得到用卸載程式所定義出來的函數。 |
| sy_spawn          | dev: sy_device_t&                                                                    | 在 dev 裝置上利用 MPI_Comm_spawn 開啟 worker 行程。                       |
| sy_open           | path: const char*<br>dev: sy_device_t<br>dl: sy_dl_t&                                | 在 dev 裝置上利用 worker 動態鏈結 path 映像檔，儲存為 dl 並回傳。                   |
| sy_close          | dev: sy_device_t<br>dl: sy_dl_t                                                      | 關閉 dev 裝置上的 dl 映像檔。                                            |
| sy_malloc         | ptr: sy_ptr_t&<br>size: size_t<br>dev: sy_device_t                                   | 在 dev 裝置上分配 size 大小的空間，並指派給 ptr。                               |
| sy_free           | ptr: sy_ptr_t<br>dev: sy_device_t                                                    | 將 dev 裝置上的 ptr 指標所指的空間釋放掉。                                     |
| sy_data_transfer  | src: sy_ptr_t<br>dst: sy_ptr_t                                                       | 將 src 指標空間上的 size 大小的資料傳送到 dst 指標空                             |

|         |                                                                                       |                                                                                      |
|---------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|         | size: size_t<br>flag: int<br>dev: sy_device_t                                         | 間中，flag 決定是主機傳送到裝置(HOST2DEVICE)還是裝置傳送到主機(DEVICE2HOST)。                               |
| sy_exec | dev: sy_device_t<br>func_addr: sy_ptr_t<br>argc: int<br>argv: void**<br>args: size_t* | 使用 ffi 函式庫呼叫 dev 裝置上 func_addr 地址上的函數，argc 為參數個數，argv 指向參數連續儲存的緩存空間，args 則記錄每個參數的大小。 |

應用程式開啟時，利用 mpirun 將行程管理與資料通訊交由 OpenMPI 系統：mpirun 會開啟所有節點上的 orted 常駐行程，orted 相互溝通並且管理行程啟動、終止等。MPI\_Send 與 MPI\_Recv 則是 OpenMPI 內的 BTL 模組負責管理，模組向使用者提供了不同的傳輸方式，相同節點上的行程利用 sm (share memory)；不同節點間則根據網路配置，可以選擇 UCX 或者 TCP 網路傳輸。

本論文網路以 USB NCM 配置，讓 USB 成為主機端與裝置端的唯一物理傳輸方式。USB NCM 會在主機端上模擬出一網路介面，並提供網路 IP 做連線。由於 OpenMPI 在 TCP 傳輸時會遍歷所有可訪問網路，在降低效能的同時還可能發生永久阻塞，因此在配置 mpirun 參數時需要設定 localhost 與該 IP 作為指定傳輸網路。

### 3.4 環境配置

本系統在裝置端使用 Buildroot 建構的簡易 Linux 系統，並且安裝 OpenMPI, OpenMP, OpenSSH, USB Gadget NCM 驅動程式以及 USB Gadget Mass Storage 驅動程式。本章節將會依序詳細敘述每個套件的安裝細節。

### 3.4.1 OpenMPI 配置

為了在裝置端支援 OpenMPI，本研究於 Buildroot 的設定過程中，透過 menuconfig 介面進行套件選擇。具體而言，可於 Target packages → Libraries → Parallel computing 類別下，勾選 OpenMPI 套件，以自動將 OpenMPI 交叉編譯進裝置端系統映像檔中。此步驟可確保裝置在啟動後，即具備執行 MPI 程式所需的函式庫與工具。然而由於 OpenMPI 要求所有節點的版本與環境對稱，但是主機端與裝置端的硬體架構有所不同，因此需要對裝置端以及主機端的 OpenMPI 做以下處理：

**裝置端：**修改 /buildroot/package/openmpi/openmpi.mk 來添加參數

```
# Disable explicitly to avoid the issue.  
OPENMPI_CONF_OPTS = --without-ime --enable-heterogeneous
```

圖 5、在 openmpi.mk 中新增 --enable-heterogeneous 開啟異構運算功能

**主機端：**根據裝置端 OpenMPI 版本下載（本論文使用 OpenMPI 4.1.5 版本），使用 ./configure 配置編譯環境，其中，OpenMPI 的依賴函式庫（hwloc, orte, pmix, libevent）需要使用 OpenMPI 內建的版本，避免使用到主機端環境下的依賴函式庫導致資料結構或函數的不相容。以 hwloc 函式庫做舉例，通過圖 3 和圖 4 的比較，可以得知 hwloc\_topology\_discovery\_support 結構體在不同版本有不同定義，若是主機端與裝置端版本不同，就會引發傳送資料長度不符預期的運行時錯誤。

```
struct hwloc_topology_discovery_support {  
    /** \brief Detecting the number of PU of  
    unsigned char pu;  
    /** \brief Detecting the number of NUMA  
    unsigned char numa;  
    /** \brief Detecting the amount of memor  
    unsigned char numa_memory;  
    /** \brief Detecting and identifying PU  
    unsigned char disallowed_pu;  
    /** \brief Detecting and identifying NUM  
    unsigned char disallowed_numa;  
    /** \brief Detecting the efficiency of C  
    unsigned char cpukind_efficiency;  
};
```

圖 6、hwloc 2.7.0 中 hwloc.h 的 hwloc\_topology\_discovery\_support 結構體

```

struct hwloc_topology_discovery_support {
    /** \brief Detecting the number of PU objects
    unsigned char pu;
    /** \brief Detecting the number of NUMA nodes
    unsigned char numa;
    /** \brief Detecting the amount of memory
    unsigned char numa_memory;
};

```

圖 7、hwloc 2.0.2 中 hwloc.h 的 hwloc\_topology\_discovery\_support 結構體

### 3.4.2 OpenMP 配置

裝置端對 OpenMP 的支援，也需透過 menuconfig 介面進行設定，可於 Toolchain → Enable OpenMP support in the toolchain 中勾選該選項，使得所產生的交叉編譯器（GCC）及標準函式庫具備對 OpenMP 編譯與執行的支援。啟用此選項後，Buildroot 將會自動配置 GCC 所需之 libgomp 函式庫，使裝置端得以正確執行包含 #pragma omp 指令的平行化程式碼。

另一方面，主機端環境則採用基於 LLVM 的工具鏈，其對 OpenMP 的支援由 libomp 提供。需注意的是，LLVM 的 libomp 與 GCC 所用的 libgomp 雖皆符合 OpenMP 標準，但於細部實作與支援範圍上仍存在差異。

為避免因 runtime 差異而導致主從系統在執行 OpenMP 程式時產生不一致行為，本研究將於實驗階段避開如 #pragma omp parallel 等部分不相容功能。

### 3.4.3 OpenSSH 配置

為使 OpenMPI 節點之間能透過 SSH 進行程序啟動與通訊，裝置端需預先整合 OpenSSH 套件。於 Buildroot 系統建構階段，可於 make menuconfig 中選擇 Target packages → Networking applications，並勾選 openssh 套件，即可在裝置系統映像中包含 sshd（伺服器端）與 ssh、scp 等用戶端工具。

在 OpenMPI 的預設設定中，若未明確指定通訊方式，將自動使用 SSH 做為節點間觸發遠端orted 程序的機制。因此，為確保 MPI 程式可於多節點之間正確啟動並通訊，需滿足以下兩項條件：

1. **SSH 無密碼登入**：各節點之間須設定 SSH 金鑰登入（例如

~/ssh/authorized\_keys)，以便主節點可不需手動輸入密碼，即可遠端啟動裝置端之 MPI 子程序。

2. **一致的使用者名稱：**OpenMPI 使用 SSH 做節點網路配置時無法使用“username@ip”格式來指定訪問節點使用者帳號，因此必須確保所有節點使用者名稱相同，本架構藉由修改 /etc/passwd 和 /etc/groups 來新增與主機端相同的使用者帳號。

#### 3.4.4 USB Gadget NCM 和 Mass Storage 驅動程式配置

為實現裝置端與主機之間的資料傳輸與網路通訊，本研究在嵌入式裝置上同時啟用 **USB Gadget NCM (Network Control Model)** 與 **Mass Storage** 模式。此功能需於 Buildroot 編譯核心前進行設定，透過 make linux-menuconfig 進入 Linux 核心配置介面，並於 Device Drivers → USB support → USB Gadget Support 中，勾選 USB Ethernet Gadget with CDC NCM support 及 Mass Storage Gadget 模組。

完成設定後，Buildroot 會將所需驅動模組整合至 Linux 核心或模組中，裝置得以於開機後載入並建立對應的 USB Gadget 介面。

為使 USB Gadget 功能於開機時自動啟用，本研究於裝置端檔案系統中的 /etc/init.d 新增自定義啟動腳本 S40usb gadget，該腳本負責設定 USB Composite Gadget 的相關參數並啟用 NCM 與 Mass Storage 功能。腳本內容如圖 5 所示，包含以下主要操作：

1. 建立 configs 所需目錄與層級（如 functions, configs, strings 等）。
2. 指定 NCM 網路介面設定。
3. 掛載對應的虛擬磁碟映像至 Mass Storage 功能（如使用 /dev/mmcbk0p1 或 loop 裝置）。
4. 將上述功能綁定至對應的 UDC 控制器。

此自動化設定使裝置在開機完成後，能即時提供主機端一組 USB 網路介面與一個可掛載的虛擬儲存裝置，便於後續進行遠端登入、檔案傳輸或其他實驗控制流程。

```
#!/bin/sh

case "$1" in
    start)
        echo "Starting USB Gadget..."

        # mount configfs
        modprobe libcomposite
        mount -t configfs none /sys/kernel/config

        # create usb gadget
        cd /sys/kernel/config/usb_gadget/
        mkdir -p sylph
        cd sylph

        # ncm
        mkdir -p functions/ncm.usb0

        # mass storage
        mkdir functions/mass_storage.usb0
        echo 1 > functions/mass_storage.usb0/lun.0/removable
        echo /mnt/usb_drive.img > functions/mass_storage.usb0/lun.0/file

        # config
        mkdir -p configs/c.1
        ln -s functions/ncm.usb0 configs/c.1/

        # mount udc
        echo "$(ls /sys/class/udc)" > UDC
        ;;

    stop)
        echo "Stopping USB Gadget..."
        ;;

    *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac

exit 0
```

圖 8、裝置端 /etc/init.d/S30usb gadget 腳本程式碼

## 第四章 實驗結果討論

本章將針對本研究所設計之異質運算架構進行實驗分析，透過多組基準測試與實際應用模型，觀察其在不同運算場景下的執行效能與資源配置表現。實驗內容涵蓋傳統多核心處理、GPU 加速與整合本研究提出之 Sylph 裝置（包含嵌入式 CPU 與 NPU）之異質運算模式，藉此驗證本系統架構於實際應用中的可行性與潛力。

本章節依實驗設計邏輯劃分為五節。第 4.1 節說明實驗所使用之硬體平台與軟體環境，包含主機與 Sylph 裝置的系統組態與開發流程。第 4.2 至 4.4 節分別使用 Rodinia benchmark、HeCBench 及 YOLOv8 作為代表性應用，對照分析多種異質運算組合下的效能表現。最後，第 4.5 節綜整各項實驗觀察，並從效能瓶頸、通訊成本與擴展性等面向進行整體討論。

除了效能數據之分析外，本研究亦深入探討實作過程中所遭遇之相容性挑戰，包含應用程式原始碼與 offload 架構之間的結構性衝突。舉例而言，Rodinia benchmark 中大量使用二維連續記憶體配置，不利於標準 map 指令進行 offload；本研究因此調整資料結構或手動使用指標區間方式進行映射。又如 YOLOv8 推論過程中，OpenCV 中大量 inline 函式無法於 LLVM toolchain 中順利鏈結，造成編譯失敗；為解決此問題，本研究自製 opencv-plugin 模組，將關鍵函式改寫為獨立函式並封裝於靜態函式庫中供鏈結使用。這些程式適配與修正工作亦構成本研究重要貢獻之一，突顯異質 offload 架構導入現有應用時所需面對的實務挑戰。

另一方面，本研究亦進一步探討資料存取策略對效能的潛在影響。由於資料搬移為異質系統中不可忽視的效能瓶頸之一，本研究在部分實驗中採用預先配置資料至 Sylph 裝置記憶體的方法，以避免重複傳輸相同資料。實驗結果顯示，此策略能有效降低通訊開銷並縮短總體執行時間，特別適用於推論類任務中模型參數等靜態資料。此發現顯示資料擺放與記憶體管理策略在異質系統效能優化中扮演關鍵角色，亦為本研究延伸之系統設計考量之一。

## 4.1 實驗環境

本研究之實驗平台建構於一組主機-裝置-NPU 三層異質架構，涵蓋高效能 x86 主機、低功耗嵌入式系統與專用神經網路處理器（NPU）之整合環境，藉以模擬實際部署中異質資源協同運算之情境。

裝置端採用 NuMaker-IoT-MA35D1-A1 開發板，搭載 ARM Cortex-A35 多核心處理器，執行由原廠提供之 Buildroot 所建構之 Linux 5.10.140 作業系統。該環境中預載支援 OpenMP 4.x 與 OpenMPI 4.1.5 之交錯平行運算套件，並搭配 OpenSSH 進行遠端控制與部署。資料通訊方面，裝置端支援 USB gadget 模式，並啟用 NCM（Network Control Model）及 Mass Storage 模式作為與主機交換資料與模擬網路的主要手段。其中，Mass Storage 模式可使裝置模擬為一個虛擬儲存裝置，供主機端掛載讀寫，NCM 模式則允許透過 USB 建立虛擬網路介面，以利 MPI 資料傳輸。

NPU 則採用 Milkv Duo 256M 模組，內部搭載 NPU 加速單元，運行同樣由原廠提供之 Buildroot 環境。該模組與 MA35D1 裝置透過網路線連接，作為推論任務的加速端點。NPU 之模型推論使用經轉換之 cvimodel 或 bmodel，並透過本研究所設計之外部 API 與 Sylph 裝置側進行整合。

主機端則為具備高效能計算能力之 Intel Core i5-12600K 架構系統，搭配 NVIDIA GeForce RTX 5070 GPU，作業系統為 Ubuntu 22.04。主機安裝 OpenMPI 4.1.5，並作為實驗控制中心與部分 benchmark 的執行載體。所有 OpenMP 與 MPI 程式均可於主機、Sylph 裝置與 NPU 間動態調度，形成完整異質協同運算鏈。

各運算元件之硬體與軟體規格彙整如下所示之表格，供後續實驗分析之參考基礎。

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |



|  |  |
|--|--|
|  |  |
|  |  |
|  |  |

## 4.2 Rodinia Benchmark

Rodinia Benchmark Suite 為一套廣泛用於異質運算研究的基準測試工具，涵蓋多種具代表性的計算密集與記憶體密集應用，包含影像處理、科學模擬、機器學習等常見場景。其設計初衷為測試不同硬體架構（如多核心 CPU、GPU、FPGA 等）在多執行緒處理下的效能行為，因此透過 OpenMP 或硬體廠商提供的 API 實作平行化。

本研究選用 Rodinia Benchmark 之主要目的，在於比較純主機端 CPU 多核心平行處理與主機 + Sylph CPU 協同處理兩種架構下的執行效能差異。為模擬儲存導向運算架構（CSD, Computational Storage Device）中資料位於裝置端的實際情境，實驗中所有輸入資料皆預先配置於 Sylph 裝置，並於執行階段透過 OpenMP target offload 技術將部分計算指派至裝置執行，藉此評估此架構對總體運算時間與通訊成本之影響。

然而，由於 Rodinia 在 OpenMP 原始程式碼是以 shared memory 模型為核心設計，其資料結構與指標使用方式不盡符合 OpenMP offload 模型對於裝置映射的語法限制。特別是在多數範例中廣泛使用之二維矩陣，以連續記憶體分配並以二重指標 (type \*\*) 操作，對應至 OpenMP target 區塊時會因 pointer aliasing 與 mapping 無法明確推導，導致無法正確將資料從裝置側映射至目標裝置。為解決此問題，本研究針對相關資料結構進行最小程度之修改，以確保不需大幅重寫卸載區塊的前提下，仍能達成主機端與裝置端之正確資料同步。以下為一個針對二維連續陣列資料所進行調整之範例程式碼（見圖 9、圖 10），展示如何透過一維指標重導與 offset 計算，使其能於 OpenMP offload 區塊中正常運作。

```
clusters = (float**) malloc(nclusters * sizeof(float*));
clusters[0] = (float*) malloc(nclusters * nfeatures * sizeof(float));
for (i=1; i<nclusters; i++)
    clusters[i] = clusters[i-1] + nfeatures;
```

圖 10、Rodinia K-Means 中二維連續陣列的定義方法

```
#ifdef OFFLOAD
float * d_clusters_buf =
    (float*)omp_target_alloc(nclusters*nfeatures*sizeof(float), dev);
float *d_clusters = omp_target_alloc(nclusters * sizeof(float), dev);
omp_target_associate_ptr(clusters, d_clusters,
    nclusters * sizeof(float*), 0, dev);
#pragma omp target is_device_ptr(d_clusters_buf)
{
    clusters[0] = d_clusters_buf;
    for (int i=1; i<nclusters; i++)
        clusters[i] = clusters[i-1] + nfeatures;
}
#endif
```

圖 11、本實驗對 Rodinia K-Means 二維連續陣列做的擴充修正

此方法之設計核心在於不修改原有應用程式邏輯與資料結構定義的前提下，透過額外的補充記憶體配置與指標初始化操作，使 OpenMP target offload 模型亦能正確詮釋並執行相關資料操作。該補充邏輯僅於初始化階段進行額外配置與關聯設定，包含：

1. 於裝置側建立與主機端相對應之線性記憶體區 (d\_clusters\_buf)；
2. 建立裝置側用於模擬指標陣列語意之記憶體區 (d\_clusters)，
3. 利用 omp\_target\_associate\_ptr 將主機端的二重指標與裝置側對應空間關聯；
4. 於 target 區塊中補充 clusters[i] = clusters[i-1] + nfeatures 的初始化動作，使其在裝置側指標語意與主機一致。

透過此補充設計，即便原始程式廣泛使用如 float\*\* 操作之結構，亦無需修改原有邏輯，即可在 OpenMP offload 執行時達成正確的資料存取與操作。此策略不僅維持應用程式之可讀性與維護性，亦大幅提升傳統共享記憶體程式碼向異質運算遷移時的適應性與可攜性，對於以既有程式碼為基礎開發之異質運算應用具高度實用價值。

本實驗採用 Rodinia Benchmark 中的 K-Means 分群演算法作為效能評估工具，該演算法主要目的是針對輸入資料計算出  $nclusters$  個群的中心點。計算流程首先將輸入資料分割為  $nthreads$  份，分派至不同核心執行，每個核心會遍歷其所屬子集，計算每筆資料與各群中心的距離，將其歸入最接近的群集，並將特徵值線性累加至對應的  $partial\_cluster\_center$ ，同時統計對應筆數  $partial\_cluster\_center\_len$ 。完成後再將所有核心的結果進行合併，計算出新的  $cluster\_center$  作為下次疊代依據，直至收斂。

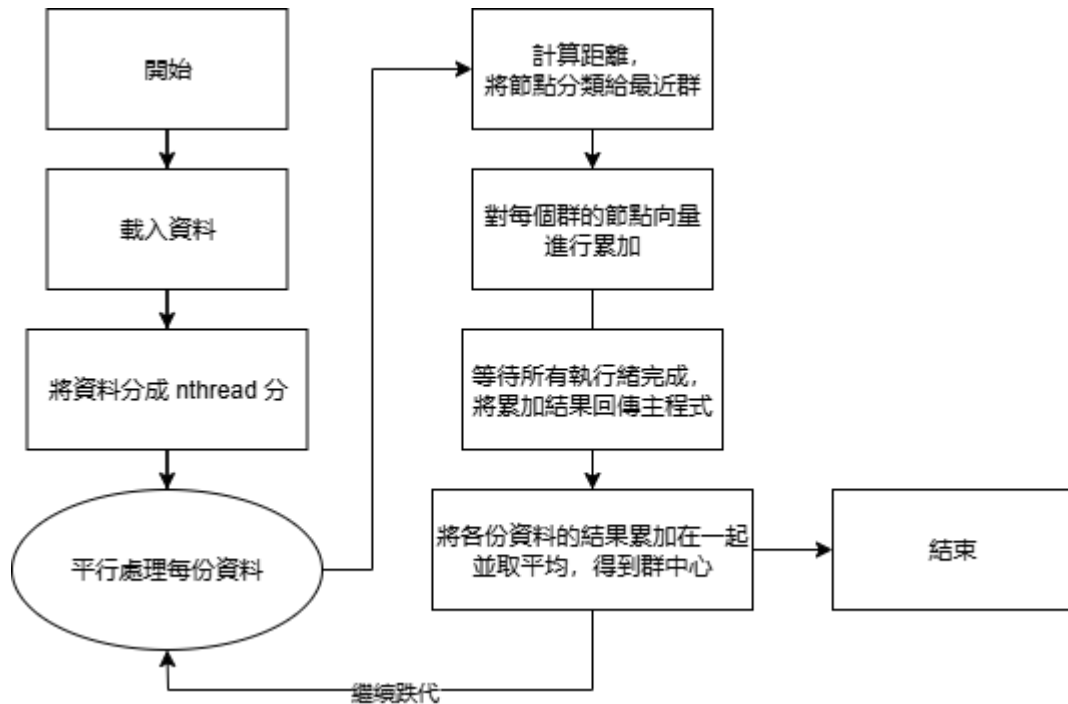


圖 12、Rodinia K-Means 分群演算法流程圖

表 a 記錄了在 OpenMP offloading 至 Sylph 裝置進行卸載計算時，各階段的資料傳輸量與執行時間。為了模擬真實 CSD 應用情境，實驗中將資料集事先儲存在 Sylph 裝置內部，主機透過 USB Mass Storage 介面存取，Sylph 裝置自身則透過 loopback driver 存取資料。比較對象為兩種實驗設計：（1）**Host 計算模式**：將資料從裝置傳回主機後，於主機執行計算；（2）**Sylph 計算模式**：將資料保留於裝置記憶體並於裝置上執行核心運算。兩者皆執行 30 次疊代，並分別記錄每次疊代中資料載入、裝置資料接收（Host-to-Device, H2D）、計算執行與結果回傳（Device-to-Host, D2H）所花費的平均時間。

表 4 顯示 10 次實驗的平均時間統計。可觀察到在 Host 計算模式中，雖然每次疊代資料讀取平均耗時高達 2643.7 ms，為 Sylph 模式的 3.74 倍（705.8 ms），但該資料僅需載入一次即可重複使用。相較之下，Sylph 模式每次疊代皆需進行一次完整的 H2D 與 D2H 資料傳輸，導致整體平均傳輸時間大幅增加，特別是 D2H 平均達 2454.7 ms，遠高於 Host 模式的 0 ms。此外，Sylph 裝置執行時間雖穩定維持在約 884 ms，但考量其 CPU 計算性能低於主機，疊代總耗時仍略遜一籌。

整體而言，儘管 Sylph 模式在資料預載入階段具備優勢，且可避免大量資料於初始階段自裝置搬移至主機，但由於每輪疊代皆需將資料結果傳回主機，且裝置執行效能受限，最終導致資料傳輸瓶頸成為性能下降的主因。本結果說明，除非能進一步減少裝置與主機間的資料交換次數（如透過完全卸載設計或在裝置端直接使用結果），否則即使裝置具備資料就近處理能力，其總體效能仍可能因傳輸開銷而受限。

表格 4、Rodinia Kmeans 實驗結果數據

| 單位 (ms)    | 資料讀取        | 平均 H2D    | 執行時間     | 平均 D2H      |
|------------|-------------|-----------|----------|-------------|
| Host 計算模式  | 2643.696875 | 0         | 22.7145  | 0           |
| Sylph 計算模式 | 705.7685    | 158.09225 | 884.1405 | 2454.707625 |

### 4.3 HeCBench

為進一步驗證本研究裝置在資料就近運算（near-data processing）情境下的實用性，本節採用 HecBench 套件進行補充測試。**HecBench** 為一組設計用於測試多核心架構下記憶體與運算密集度差異的 benchmark 程式，其涵蓋如 stencil、attention 等典型運算模型，並支援 OpenMP target offload，適合分析異質裝置在實際 workload 下之效能表現。

根據前一節 Rodinia Benchmark 的實驗結論顯示，若裝置端於每次疊代中皆需進行高頻率資料傳輸，則即便計算具備平行性優勢，仍會受限於傳輸開銷。因此，本節特別挑選 HecBench 中資料傳輸量較小、計算較密集的 attention 測試進行實驗，以探討當每輪疊代所需資料載入量有限時，裝置是否能有效發揮其近資料運算之優勢。

attention 測試模擬神經網路中的注意力機制（Attention Mechanism），以多次疊代（Epoch）進行矩陣乘法與 Softmax 運算，其計算密集、記憶體佔用可隨資料集大小調整，適合作為本研究裝置在迭代式任務中的表現評估依據。圖 13 為 Attention 測驗的運算流程圖，其中三個疊代內的矩陣運算為三個獨立的卸載工作，並依序由主機端呼叫，運算期間會使用已經傳送到裝置上的資料，所以不會進行資料傳遞。

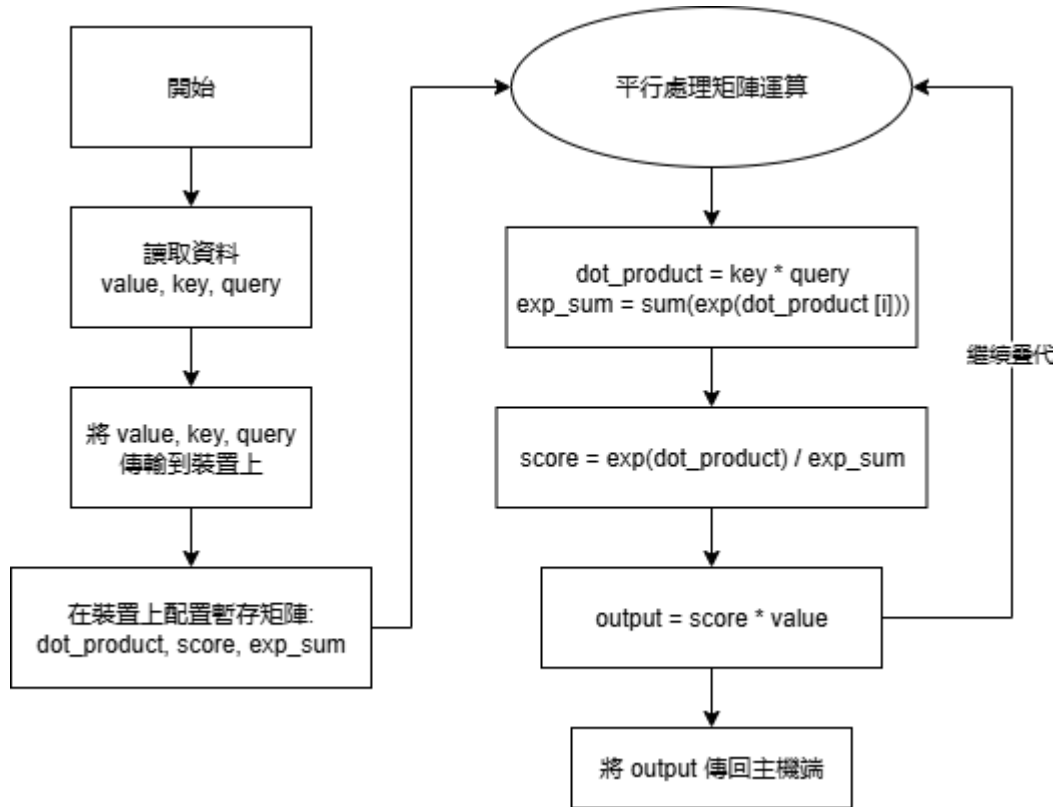


圖 13、HeCBench Attention 演算法流程

本次測試設定疊代次數（Epoch）為 50，並逐步調整輸入資料大小，以觀察主機與裝置在不同資料規模下的效能變化。attention 測試程式中之輸入資料由三個矩陣構成：value 與 key 為大小為  $n \times d$  的浮點數矩陣，query 為大小為  $1 \times d$  的浮點數矩陣。為維持運算維度穩定，本實驗將特徵維度  $d$  固定為 1024，並依據測試輪次動態調整樣本數量  $n$ ，使資料總量隨測試次數呈倍增成長。

具體而言，本實驗將  $n$  設定為  $n = 256 \times 2^k$ ，其中  $k \in \{0, 1, 2, \dots, 7\}$ ，對應產生從數 MB 到數百 MB 不等的輸入資料規模，最終產生之測試資料總大小與各階段耗時結果整理如表 5 所示。

表格 5、HeCBench Attention 實驗數據

| 總資料大小(bytes) | 裝置讀取(us)   | 裝置執行(us)      | 主機讀取(us)   | 主機執行(us) |
|--------------|------------|---------------|------------|----------|
| 4,196,360    | 71,644     | 5,937,134     | 168,287    | 6,295    |
| 8,392,712    | 105,015    | 12,407,668    | 338,811    | 7,839    |
| 16,781,320   | 177,540    | 24,645,750    | 671,377    | 23,209   |
| 33,558,536   | 555,223    | 49,307,529    | 1,335,721  | 36,370   |
| 67,112,968   | 1,190,076  | 98,428,017    | 2,628,970  | 96,450   |
| 134,221,832  | 2,125,917  | 196,344,839   | 5,047,259  | 193,846  |
| 268,439,560  | 4,289,413  | 395,165,028   | 8,981,833  | 397,309  |
| 536,875,016  | 51,221,976 | 1,851,081,721 | 18,127,613 | 779,923  |

## 4.4 YOLOv8：推論任務在多元加速器下的比較

根據前述各項實驗結果可觀察到，雖然透過將資料預先儲存於 Sylph 裝置可有效降低資料傳輸延遲，進而縮短總體通訊成本，然而由於 Sylph 採用之 ARM Cortex-A35 處理器計算能力相較主機端 CPU 明顯不足，即使具備 offload 平行執行能力，整體執行時間仍往往高於主機單純使用多核心或 GPU 加速之情境。因此，僅以 Sylph CPU 作為 offload 計算資源，在部分應用場景中仍難以達到可接受之效能表現。

為進一步探討 Sylph 架構在特定應用中是否具備實質效能優勢，本節針對深度學習模型 YOLOv8 進行推論任務測試，並聚焦於當 Sylph 可調用其內部 NPU 加速單元時，是否能顯著降低推論時間與系統負載。本實驗將 YOLOv8 推論流程區分為兩大階段：資料讀取和執行。所有輸入資料皆統一儲存在 Sylph 端，以確保公平比較傳輸與處理開銷。

在實作過程中，亦發現部分 OpenCV 函式在 LLVM OpenMP offload 模型中存在相容性問題。由於 OpenCV 許多常用函式（如影像縮放、轉換等）係以 inline function 定義於標頭檔內，LLVM Clang 編譯器於處理 #pragma omp target 區塊時並不

會自動攜帶主程式中的 inline 函式定義，導致卸載程式於編譯階段無法解析對應實作而產生 undefined reference 錯誤。為解決此問題，本研究額外實作一套 libopencv-plugin.so 函式庫，將關鍵 inline 函式重新封裝為一般外部函式，並於卸載模組鏈結階段導入對應實作。

## 4.5 整體比較與分析

# 第五章 結論與未來展望

## 5.1 結論

## 5.2 研究限制

## 5.3 未來研究方向

## 參考文獻

- [1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, “AI and Memory Wall,” *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024, doi: 10.1109/MM.2024.3373763.
- [2] J. Zhang and M. Jung, “ZnG: Architecting GPU Multi-Processors with New Flash for Scalable Data Analysis,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, Valencia, Spain: IEEE, May 2020, pp. 1064–1075. doi: 10.1109/ISCA45697.2020.00090.
- [3] H. Zhang, Y. Zhou, Y. Xue, Y. Liu, and J. Huang, “G10: Enabling An Efficient Unified GPU Memory and Storage Architecture with Smart Tensor Migrations,” in *56th Annual IEEE/ACM International Symposium on Microarchitecture*, Toronto ON Canada: ACM, Oct. 2023, pp. 395–410. doi: 10.1145/3613424.3614309.
- [4] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, “ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning,” Apr. 16, 2021, *arXiv*: arXiv:2104.07857. doi: 10.48550/arXiv.2104.07857.
- [5] A. Barbalace and J. Do, “Computational Storage: Where Are We Today?”.
- [6] “The LLVM Compiler Infrastructure Project.”
- [7] A.-L. Varbanescu, A. Bhatele, P. Luszczek, and B. Marc, Eds., *High Performance Computing: 37th International Conference, ISC High Performance 2022, Hamburg, Germany, May 29 – June 2, 2022, Proceedings*. in Lecture Notes in Computer Science. Cham: Springer International Publishing, 2022. doi: 10.1007/978-3-031-07312-0.
- [8] B. Shan, M. Araya-Polo, A. M. Malik, and B. Chapman, “MPI-based Remote OpenMP Offloading: A More Efficient and Easy-to-use Implementation,” in *Proceedings of the 14th International Workshop on Programming Models and Applications for Multicores and Manycores*, Montreal QC Canada: ACM, Feb. 2023, pp. 50–59. doi: 10.1145/3582514.3582519.
- [9] P. Walia, I. Shanware, K. Vaidyanathan, D. D. Kalamkar, and U. M. Natarajan, “LOSM: Leveraging OpenMP and Shared Memory for Accelerating Blocking MPI Allreduce,” in *2024 IEEE 31st International Conference on High Performance Computing, Data and Analytics Workshop (HiPCW)*, Bangalore, India: IEEE, Dec. 2024, pp. 123–124. doi: 10.1109/hipcw63042.2024.00039.
- [10] S. Alam, C. Yakopcic, Q. Wu, M. Barnell, S. Khan, and T. M. Taha, “Survey of Deep Learning Accelerators for Edge and Emerging Computing,” *Electronics*, vol. 13, no. 15, p. 2988, Jul. 2024, doi: 10.3390/electronics13152988.
- [11] S. Liang and Y. Wang, “Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval”.